Training neural networks with PyTorch

Weekend 1, Saturday morning

Carlos Cotrini

ETH Zurich

Saturday 5 September 2026

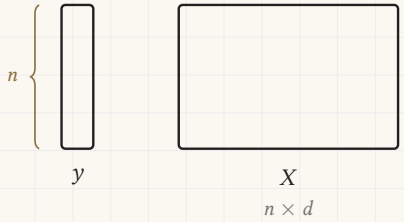
1

Section 1

What Friday’s loop costs

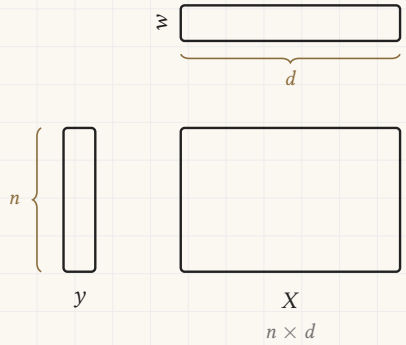
The picture from Friday afternoon

Friday afternoon, unchanged. Each box is a piece of data, and its shape is the shape of the data.



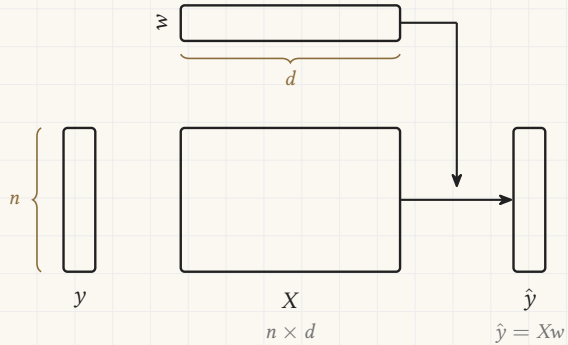
The picture from Friday afternoon

Friday afternoon, unchanged. Each box is a piece of data, and its shape is the shape of the data.



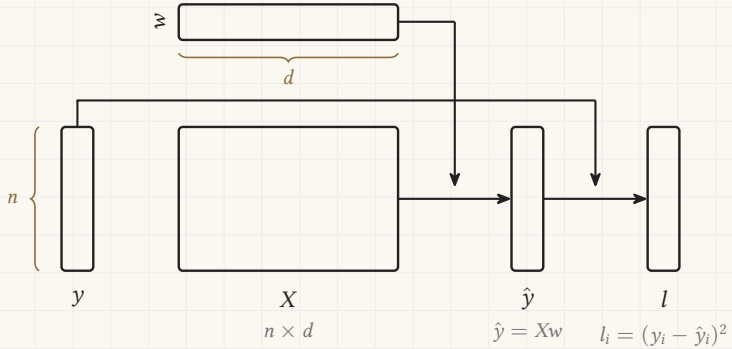
The picture from Friday afternoon

Friday afternoon, unchanged. Each box is a piece of data, and its shape is the shape of the data.



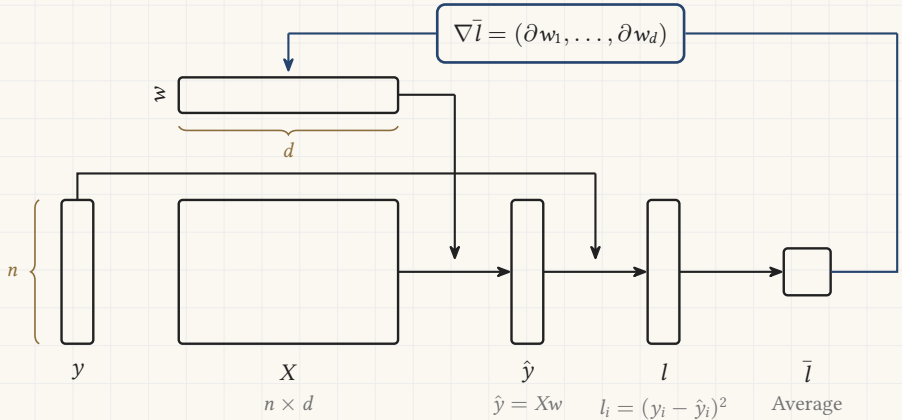
The picture from Friday afternoon

Friday afternoon, unchanged. Each box is a piece of data, and its shape is the shape of the data.



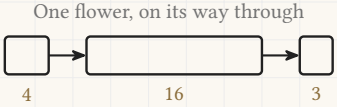
The picture from Friday afternoon

Friday afternoon, unchanged. Each box is a piece of data, and its shape is the shape of the data.



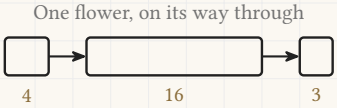
The numbers this network holds

Four measurements in, sixteen hidden values in the middle, three species scores out.



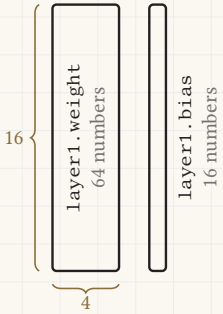
The numbers this network holds

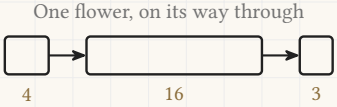
Four measurements in, sixteen hidden values in the middle, three species scores out.



The numbers this network holds

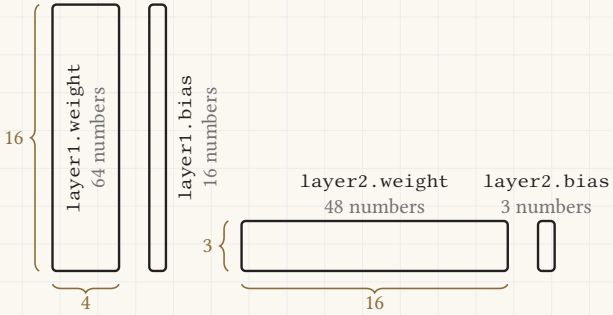
Four measurements in, sixteen hidden values in the middle, three species scores out.

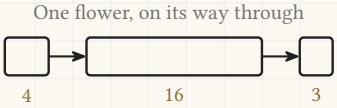




The numbers this network holds

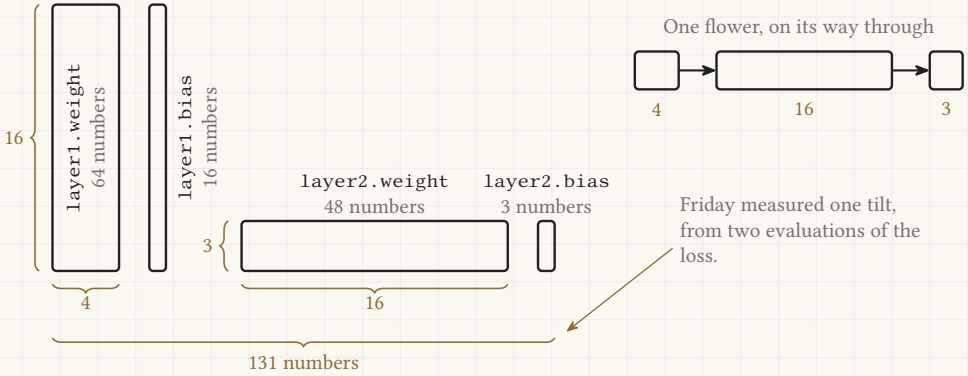
Four measurements in, sixteen hidden values in the middle, three species scores out.





The numbers this network holds

Four measurements in, sixteen hidden values in the middle, three species scores out.



What replaces what

The data, as a block of numbers	`torch.tensor(...)`

What replaces what

The data, as a block of numbers	`torch.tensor(...)`
The network of boxes	`nn.Linear, nn.ReLU, nn.Module`

What replaces what

The data, as a block of numbers	`torch.tensor(...)`
The network of boxes	`nn.Linear, nn.ReLU, nn.Module`
The verdict, how wrong we are	`nn.CrossEntropyLoss()`

What replaces what

The data, as a block of numbers	`torch.tensor(...)`
The network of boxes	`nn.Linear, nn.ReLU, nn.Module`
The verdict, how wrong we are	`nn.CrossEntropyLoss()`
The tilt, measured in a window	`loss.backward()`

What replaces what

The data, as a block of numbers	`torch.tensor(...)`
The network of boxes	`nn.Linear, nn.ReLU, nn.Module`
The verdict, how wrong we are	`nn.CrossEntropyLoss()`
The tilt, measured in a window	`loss.backward()`
`w = w - alpha * g`	`optimizer.step()`

What replaces what

The data, as a block of numbers	`torch.tensor(...)`
The network of boxes	`nn.Linear, nn.ReLU, nn.Module`
The verdict, how wrong we are	`nn.CrossEntropyLoss()`
The tilt, measured in a window	`loss.backward()`
`w = w - alpha * g`	`optimizer.step()`
The step size, α	`lr=0.05`

2

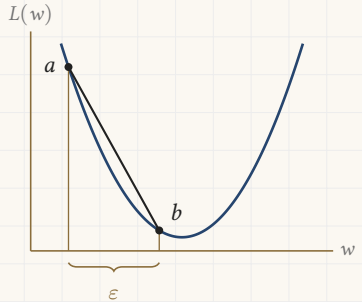
Section 2

The gradient and the update

The gradient, for every weight at once

Friday, 11:00. Measuring the tilt of one weight.

```
a = L(w - eps/2)
b = L(w + eps/2)
g = (b - a) / eps
```

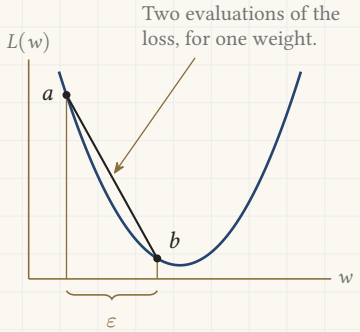


One weight

The gradient, for every weight at once

Friday, 11:00. Measuring the tilt of one weight.

```
a = L(w - eps/2)
b = L(w + eps/2)
g = (b - a) / eps
```



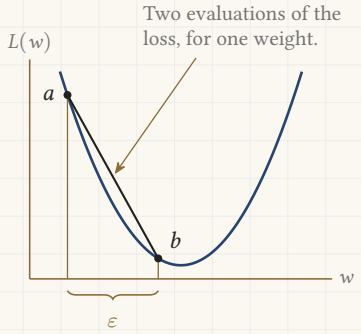
One weight

The gradient, for every weight at once

Friday, 11:00. Measuring the tilt of one weight.

```
a = L(w - eps/2)
b = L(w + eps/2)
g = (b - a) / eps

loss.backward()
```



One weight

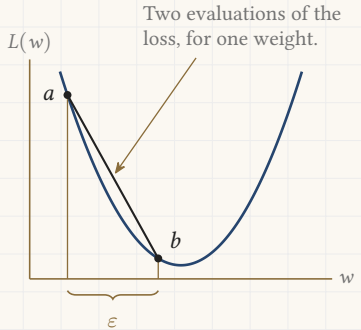
The gradient, for every weight at once

Friday, 11:00. Measuring the tilt of one weight.

~~a = L(w - eps/2)~~
~~b = L(w + eps/2)~~
~~g = (b - a) / eps~~

```
loss.backward()
```

One call, and every one of the 131 numbers has its own tilt waiting on it.



One weight

The update rule, in one call

The step Friday took by hand, once for each weight.

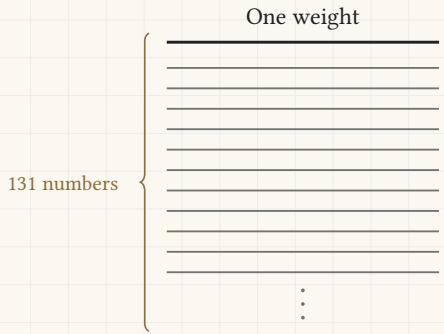
```
w = w - alpha * g
```

One weight

The update rule, in one call

The step Friday took by hand, once for each weight.

```
w = w - alpha * g
```

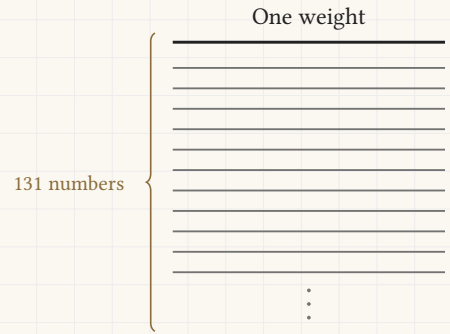


The update rule, in one call

The step Friday took by hand, once for each weight.

```
w = w - alpha * g
```

```
optimizer.step()
```

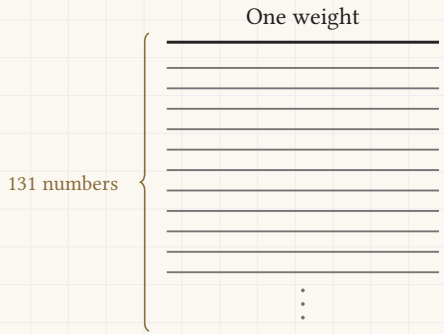


The update rule, in one call

The step Friday took by hand, once for each weight.

```
w = w - alpha * g
```

```
optimizer.step()
```



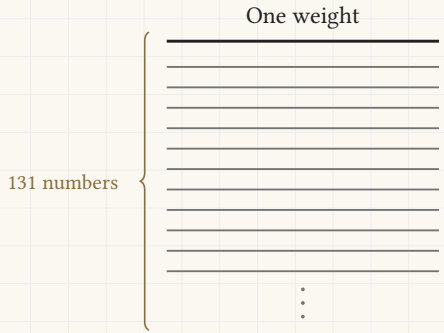
The update rule, in one call

The step Friday took by hand, once for each weight.

~~`w = w - alpha * g`~~

```
optimizer.step()
```

One call moves all 131.



3

Section 3

The notebook

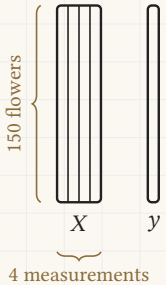
The data

The Iris measurements, and the two blocks every later frame is drawn from.


```
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

The data

The Iris measurements, and the two blocks every later frame is drawn from.

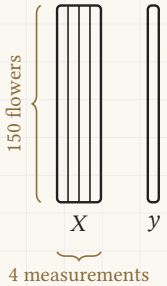



```
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data
y = iris.target
```

The data

The Iris measurements, and the two blocks every later frame is drawn from.



X Sepal length
Sepal width
Petal length
Petal width

y Setosa
Versicolor
Virginica

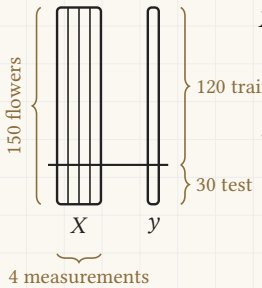
```
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data
y = iris.target

print("Features shape:", X.shape)
print("Labels shape: ", y.shape)
print("Classes:      ", iris.target_names)
```

The data

The Iris measurements, and the two blocks every later frame is drawn from.



X Sepal length
Sepal width
Petal length
Petal width

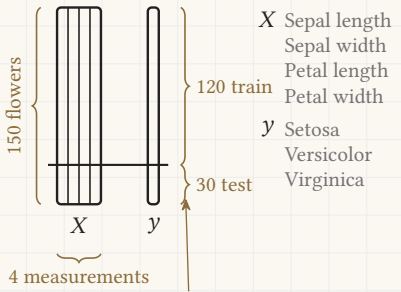
y Setosa
Versicolor
Virginica

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test  = torch.tensor(X_test,  dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test  = torch.tensor(y_test,  dtype=torch.long)
```

The data

The Iris measurements, and the two blocks every later frame is drawn from.



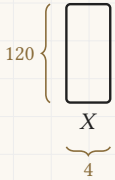

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test  = torch.tensor(X_test,  dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test  = torch.tensor(y_test,  dtype=torch.long)
```

The network, as blocks

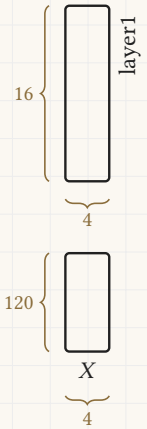
The same picture as yesterday, with the notebook's own widths.

```
import torch.nn as nn
```



The network, as blocks

The same picture as yesterday, with the notebook's own widths.

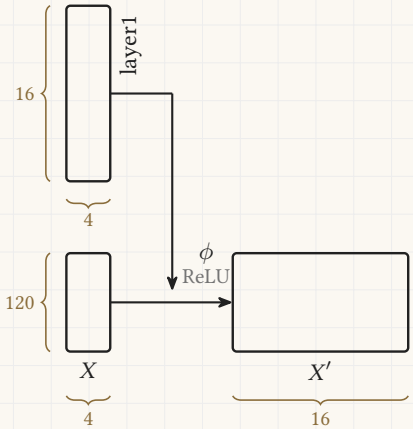



```
import torch.nn as nn

self.layer1 = nn.Linear(4, 16)
```

The network, as blocks

The same picture as yesterday, with the notebook's own widths.

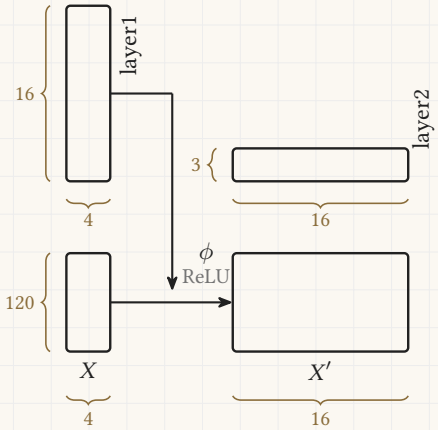



```
import torch.nn as nn

self.layer1 = nn.Linear(4, 16)
self.relu   = nn.ReLU()
```

The network, as blocks

The same picture as yesterday, with the notebook's own widths.

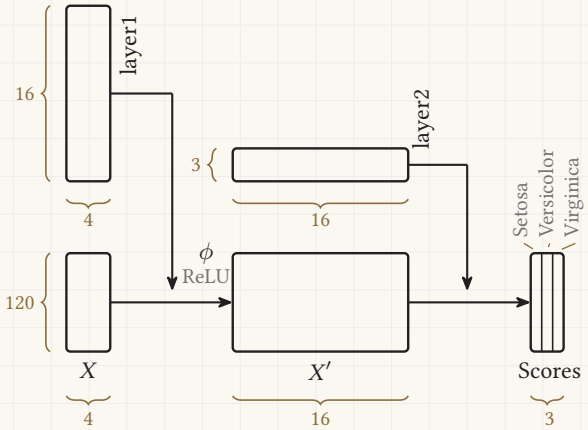



```
import torch.nn as nn

self.layer1 = nn.Linear(4, 16)
self.relu   = nn.ReLU()
self.layer2 = nn.Linear(16, 3)
```

The network, as blocks

The same picture as yesterday, with the notebook's own widths.




```
import torch.nn as nn

self.layer1 = nn.Linear(4, 16)
self.relu   = nn.ReLU()
self.layer2 = nn.Linear(16, 3)
```

The network, as a class

The parts

The wiring

```
class IrisNet(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
```

The network, as a class

The parts

The wiring

```
class IrisNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(4, 16)
        self.relu   = nn.ReLU()
        self.layer2 = nn.Linear(16, 3)

    def forward(self, x):
```

The network, as a class

The parts

```
class IrisNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(4, 16)
        self.relu   = nn.ReLU()
        self.layer2 = nn.Linear(16, 3)
```

The wiring

```
    def forward(self, x):
        x = self.layer1(x)      # 1
        x = self.relu(x)        # 2
        x = self.layer2(x)      # 3
        return x
```

The network, as a class

The parts

The wiring

```
class IrisNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(4, 16)
        self.relu   = nn.ReLU()
        self.layer2 = nn.Linear(16, 3)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
```

1 2 3

```
model = IrisNet()
```

The network, as a class

The parts

```
class IrisNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(4, 16)
        self.relu   = nn.ReLU()
        self.layer2 = nn.Linear(16, 3)
```

The wiring

1
2
3

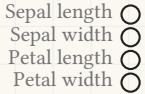
```
    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
```

```
model = IrisNet()
print(model)
```

```
IrisNet(
  (layer1): Linear(in_features=4, out_features=16, bias=True)
  (relu): ReLU()
  (layer2): Linear(in_features=16, out_features=3, bias=True)
)
```

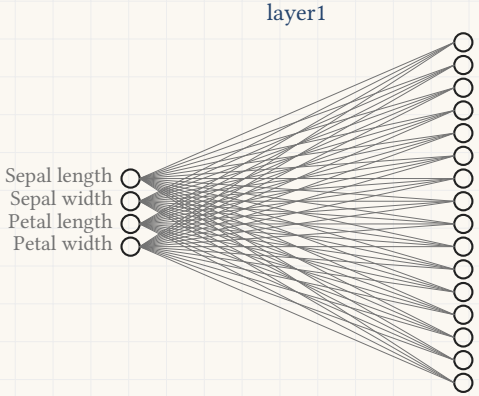
The network, drawn as neurons

One circle for every column of every weight block.



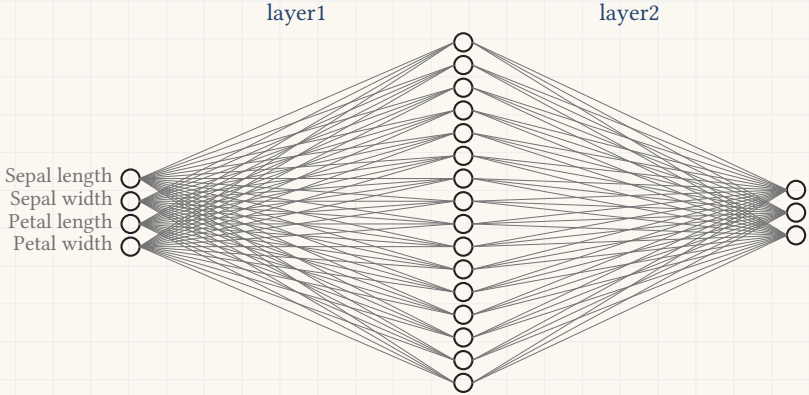
The network, drawn as neurons

One circle for every column of every weight block.



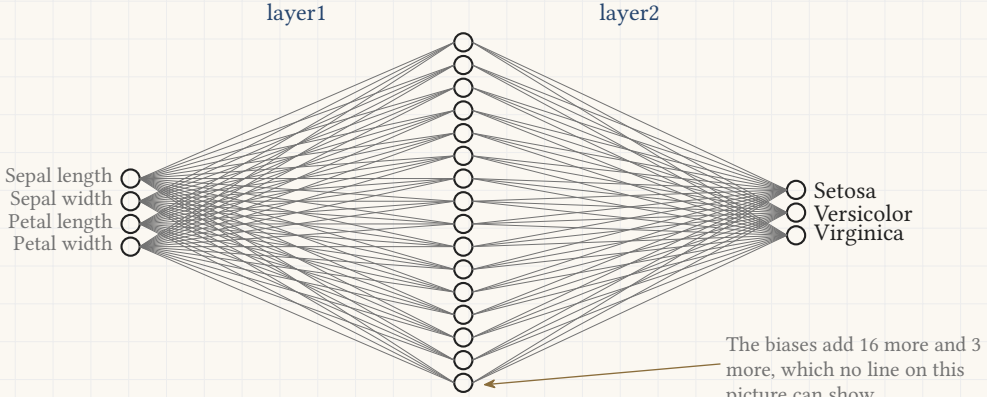
The network, drawn as neurons

One circle for every column of every weight block.



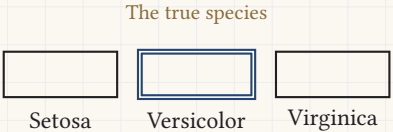
The network, drawn as neurons

One circle for every column of every weight block.



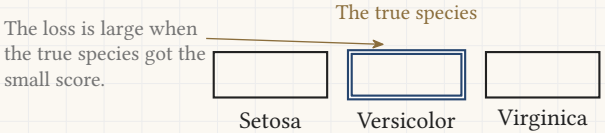
The loss and the optimizer

Friday's miss measured a number. This one scores a choice among three species.



The loss and the optimizer

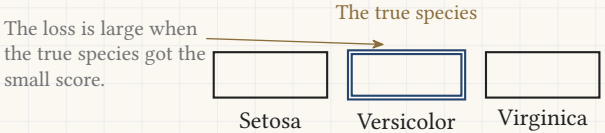
Friday's miss measured a number. This one scores a choice among three species.




```
loss_fn    = nn.CrossEntropyLoss()
```

The loss and the optimizer

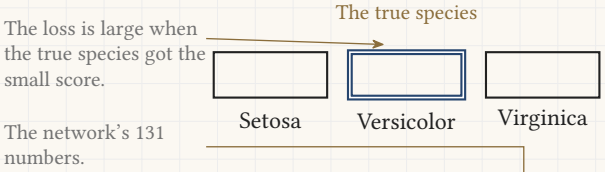
Friday's miss measured a number. This one scores a choice among three species.




```
loss_fn   = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
```

The loss and the optimizer

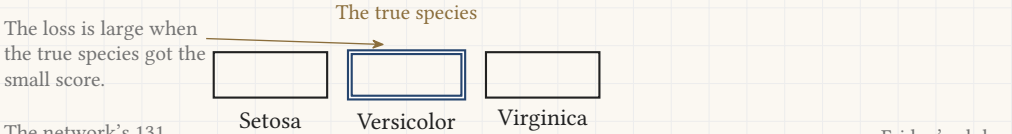
Friday's miss measured a number. This one scores a choice among three species.




```
loss_fn   = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
```

The loss and the optimizer

Friday's miss measured a number. This one scores a choice among three species.



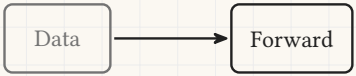

```
loss_fn   = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
```

The training loop


```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
```

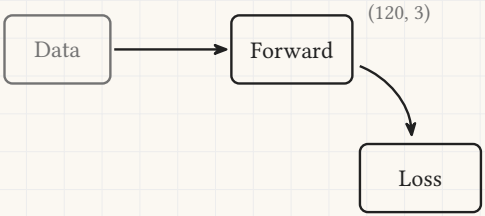
The training loop




```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
    predictions = model(X_train)
```

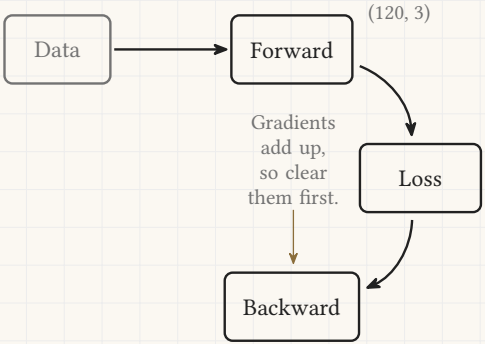
The training loop




```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
```

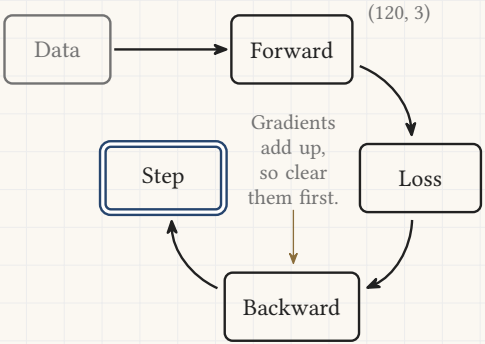
The training loop




```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
    optimizer.zero_grad()
    loss.backward()
```

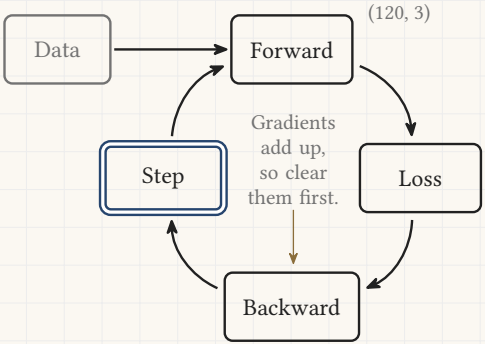
The training loop




```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

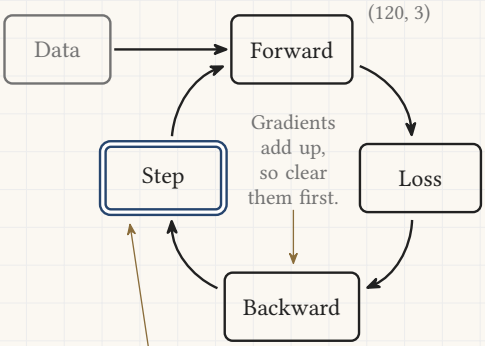
The training loop




```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    loss_history.append(loss.item())
```

The training loop

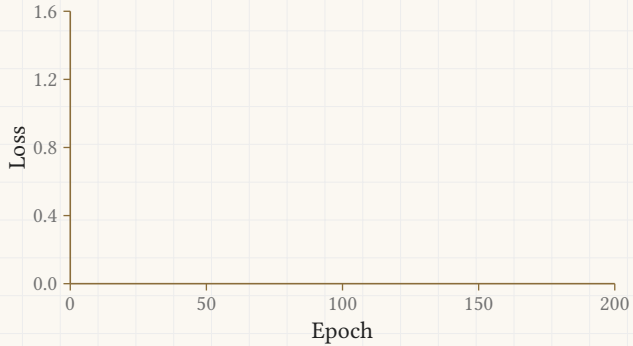



```
num_epochs = 200
loss_history = []

for epoch in range(num_epochs):
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    loss_history.append(loss.item())
```

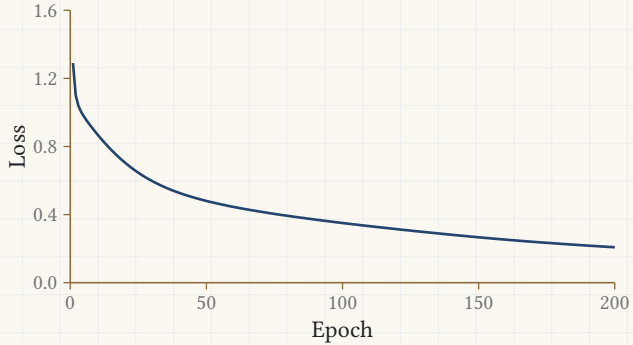
The learning curve

One seeded run. Your own run will differ.



The learning curve

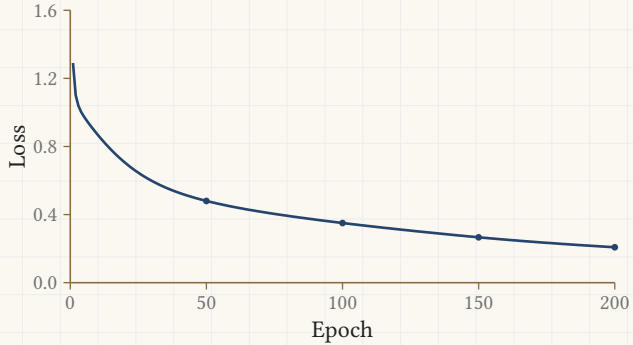
One seeded run. Your own run will differ.




```
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

The learning curve

One seeded run. Your own run will differ.

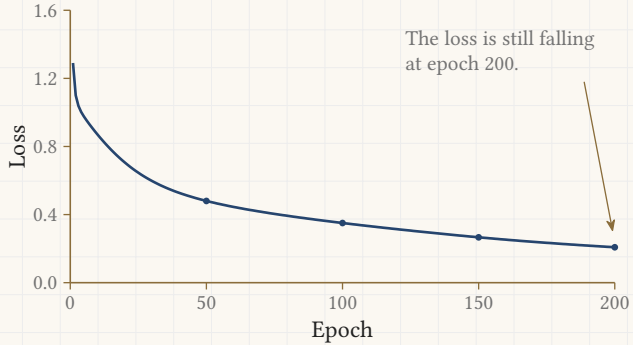



```
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

```
Epoch  50 / 200  |  Loss: 0.4802
Epoch 100 / 200  |  Loss: 0.3505
Epoch 150 / 200  |  Loss: 0.2665
Epoch 200 / 200  |  Loss: 0.2082
```

The learning curve

One seeded run. Your own run will differ.




```
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

```
Epoch  50 / 200  |  Loss: 0.4802
Epoch 100 / 200  |  Loss: 0.3505
Epoch 150 / 200  |  Loss: 0.2665
Epoch 200 / 200  |  Loss: 0.2082
```

The test set

One seeded run. Your own run will differ.

```
with torch.no_grad():
    test_scores = model(X_test)
    predicted   = test_scores.argmax(1)
```

The test set

One seeded run. Your own run will differ.

```
with torch.no_grad():
    test_scores = model(X_test)
    predicted   = test_scores.argmax(1)
```

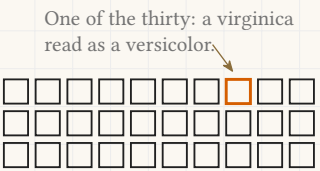
The call `argmax(1)` keeps the species with the largest score.

The test set

One seeded run. Your own run will differ.

```
with torch.no_grad():
    test_scores = model(X_test)
    predicted   = test_scores.argmax(1)
```

The call `argmax(1)` keeps the species with the largest score.



The test set

One seeded run. Your own run will differ.

```
with torch.no_grad():
    test_scores = model(X_test)
    predicted   = test_scores.argmax(1)
```

The call `argmax(1)` keeps the species with the largest score.

```
correct  = (predicted == y_test).sum().item()
total    = y_test.shape[0]
accuracy = correct / total * 100
```

One of the thirty: a virginica read as a versicolor.

The test set

One seeded run. Your own run will differ.

```
with torch.no_grad():
    test_scores = model(X_test)
    predicted   = test_scores.argmax(1)
```

The call `argmax(1)` keeps the species with the largest score.

```
correct  = (predicted == y_test).sum().item()
total    = y_test.shape[0]
accuracy = correct / total * 100

print(f"Correct: {correct} / {total}")
print(f"Accuracy: {accuracy:.1f}%")
```

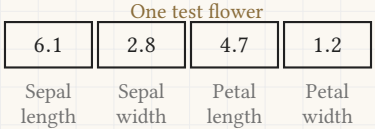
One of the thirty: a virginica read as a versicolor.

```
Correct: 29 / 30
Accuracy: 96.7%
```

One prediction up close

One seeded run. Your own run will differ.

```
flower = X_test[0].unsqueeze(0)
true_label = y_test[0].item()
```

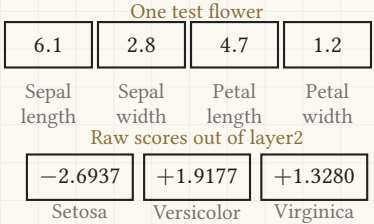


One prediction up close

One seeded run. Your own run will differ.

```
flower = X_test[0].unsqueeze(0)
true_label = y_test[0].item()

with torch.no_grad():
    scores = model(flower)
```



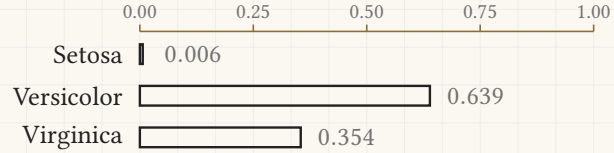
One prediction up close

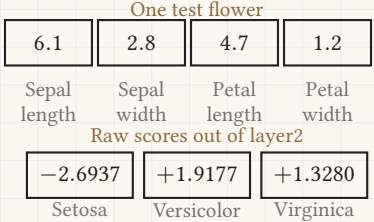
One seeded run. Your own run will differ.

```
flower = X_test[0].unsqueeze(0)
true_label = y_test[0].item()

with torch.no_grad():
    scores = model(flower)

probs = torch.softmax(scores, dim=1)[0]
```





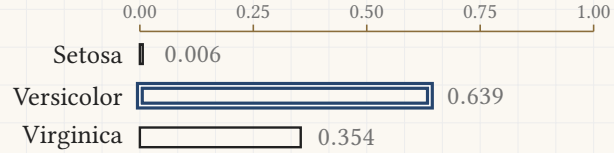
One prediction up close

One seeded run. Your own run will differ.

```
flower = X_test[0].unsqueeze(0)
true_label = y_test[0].item()

with torch.no_grad():
    scores = model(flower)

probs = torch.softmax(scores, dim=1)[0]
```



One test flower

6.1	2.8	4.7	1.2
Sepal length	Sepal width	Petal length	Petal width

Raw scores out of layer2

−2.6937	+1.9177	+1.3280
Setosa	Versicolor	Virginica

```
True species:    versicolor
```

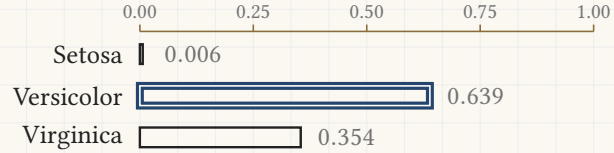
One prediction up close

One seeded run. Your own run will differ.

```
flower = X_test[0].unsqueeze(0)
true_label = y_test[0].item()

with torch.no_grad():
    scores = model(flower)

probs = torch.softmax(scores, dim=1)[0]
```



The network returns these three raw scores. The softmax runs here, once, after training, so the numbers can be read as probabilities.

One test flower

6.1	2.8	4.7	1.2
Sepal length	Sepal width	Petal length	Petal width

Raw scores out of layer2

−2.6937	+1.9177	+1.3280
Setosa	Versicolor	Virginica

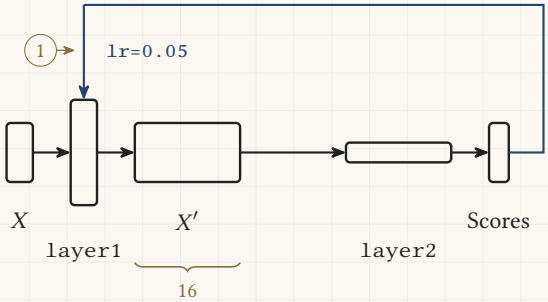
True species: versicolor

4

Section 4

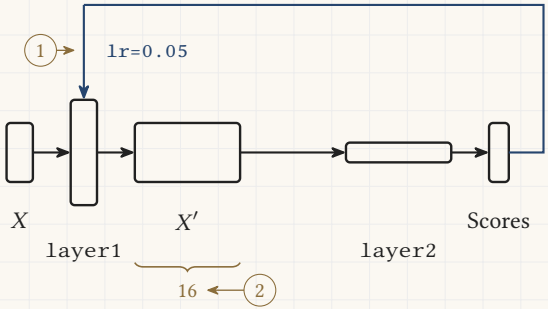
The knobs

Four things to change



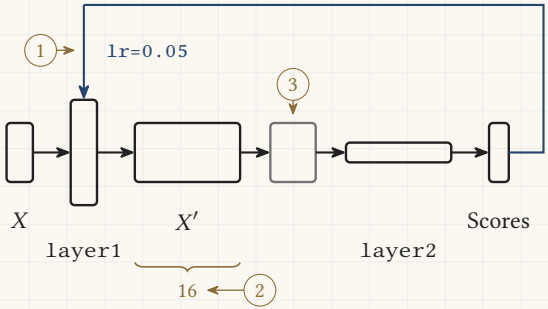
1. Change the step: try 0.1 or 0.01.

Four things to change



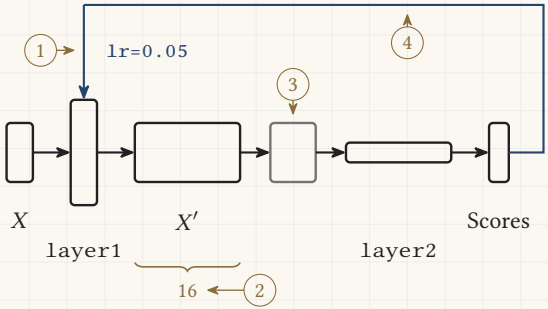
1. Change the step: try 0.1 or 0.01.
2. Change the hidden layer: 16 becomes 4 or 64.

Four things to change



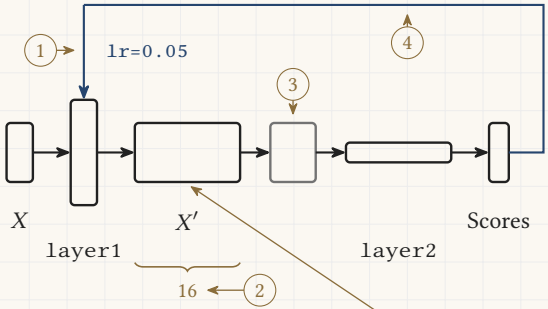
1. Change the step: try 0.1 or 0.01.
2. Change the hidden layer: 16 becomes 4 or 64.
3. Add a second hidden layer.

Four things to change



1. Change the step: try 0.1 or 0.01.
2. Change the hidden layer: 16 becomes 4 or 64.
3. Add a second hidden layer.
4. Train for more epochs, or fewer.

Four things to change



1. Change the step: try 0.1 or 0.01.
2. Change the hidden layer: 16 becomes 4 or 64.
3. Add a second hidden layer.
4. Train for more epochs, or fewer.

What replaced what

The data, as a block of numbers	
The network of boxes	
The verdict, how wrong we are	
The tilt, measured in a window	
`w = w - alpha * g`	
The step size, α	

What replaced what

The data, as a block of numbers	`torch.tensor(...)`
The network of boxes	`nn.Linear, nn.ReLU, nn.Module`
The verdict, how wrong we are	`nn.CrossEntropyLoss()`
The tilt, measured in a window	`loss.backward()`
`w = w - alpha * g`	`optimizer.step()`
The step size, α	`lr=0.05`